

Scaling LLM Test-Time Compute with Mobile NPU on Smartphones

Zixu Hao
Tsinghua University
haozx23@mails.tsinghua.edu.cn

Jianyu Wei
University of Science and Technology
of China
noob@mail.ustc.edu.cn

Tuowei Wang
Tsinghua University
wtw23@mails.tsinghua.edu.cn

Minxing Huang
Tsinghua University
huangmx25@mails.tsinghua.edu.cn

Huiqiang Jiang
Microsoft Research
hjiang@microsoft.com

Shiqi Jiang
Microsoft Research
shijiang@microsoft.com

Ting Cao*
Institute for AI Industry Research
(AIR), Tsinghua University
tingcao@mail.tsinghua.edu.cn

Ju Ren*
Tsinghua University
renju@tsinghua.edu.cn

Abstract

Deploying Large Language Models (LLMs) on mobile devices faces the challenge of insufficient performance in smaller models and excessive resource consumption in larger ones. This paper highlights that mobile Neural Processing Units (NPUs) have underutilized computational resources, particularly their matrix multiplication units, during typical LLM inference. To leverage this wasted compute capacity, we propose applying parallel test-time scaling techniques on mobile NPUs to enhance the performance of smaller LLMs. However, this approach confronts inherent NPU challenges, including inadequate hardware support for fine-grained quantization and low efficiency in general-purpose computations. To overcome these, we introduce two key techniques: a hardware-aware tile quantization scheme that aligns group quantization with NPU memory access patterns, and efficient LUT-based replacements for complex operations such as Softmax and dequantization. We design and implement an end-to-end inference system that leverages the NPU's compute capability to support test-time scaling on Qualcomm Snapdragon platforms. Experiments show our approach brings significant speedups: up to 19.0× for mixed-precision GEMM and 2.2× for Softmax. More importantly, we demonstrate that smaller models using test-time scaling can match or exceed the accuracy of larger models, achieving a new performance-cost Pareto frontier.

*Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.

EUROSYS '26, April 27–30, 2026, Edinburgh, Scotland UK

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/26/04

<https://doi.org/10.1145/3767295.3769382>

CCS Concepts: • Computer systems organization → Single instruction, multiple data; • Human-centered computing → Ubiquitous and mobile devices; • Computing methodologies → Natural language processing.

Keywords: Neural Processing Unit, mobile device, Large Language Model

ACM Reference Format:

Zixu Hao, Jianyu Wei, Tuowei Wang, Minxing Huang, Huiqiang Jiang, Shiqi Jiang, Ting Cao, and Ju Ren. 2026. Scaling LLM Test-Time Compute with Mobile NPU on Smartphones. In *European Conference on Computer Systems (EUROSYS '26)*, April 27–30, 2026, Edinburgh, Scotland UK. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3767295.3769382>

1 Introduction

With the advancements of commodity hardware and algorithms, deploying Large Language Models (LLMs) on mobile devices is becoming increasingly feasible. Many language models tailored for mobile devices have emerged, including Llama 3.2 [40], MiniCPM [22, 65], Gemma [51, 52]. However, these models generally underperform compared to their larger counterparts. A straightforward approach to improve model performance is to scale up the model size, yet this significantly increases memory consumption and bandwidth requirements, posing serious challenges for resource-constrained mobile platforms.

Recently, a new paradigm named **test-time scaling** have introduced new opportunities to enhance LLM capabilities through increased inference-time computation. Parallel test-time scaling methods involve generating multiple paths and selecting the best sample among a number of generation candidates [3, 4, 23, 35, 50, 55, 57, 60]. So far, these methods are limited to cloud or offline settings where computational resources are abundant.

Intuitively, employing test-time scaling techniques to enhance LLM’s generation quality on mobile devices may seem impractical. Mobile devices such as smartphones are typically considered resource-constrained, while LLM inference is known for its high resource consumption. On top of this, scaling compute resources at runtime requires even more computation.

However, recent integration of Neural Processing Units (NPU) in mobile SoCs has begun to shift this landscape. Vendors including Qualcomm, Intel, and AMD have designed and integrated NPUs to accelerate AI workloads [9, 39, 48]. These NPUs not only achieve high peak computing power but also undergo rapid evolution: Qualcomm claims that its Hexagon NPU in Snapdragon X Elite delivers 45 TOPS of INT8 performance [25], while recent generations of AMD NPUs have achieved $3.1\times^1$ performance improvements [24]. These developments are transforming the computation capabilities of mobile devices.

We discover that mobile NPUs achieve high peak performance through dedicated matrix multiplication units that operate on large matrix tiles. However, in typical LLM inference, GEMM operations often degenerate into GEMV during the decoding phase, resulting in low hardware utilization and waste of computing capabilities of the large-tile optimized matrix units. This underutilization presents an opportunity: test-time scaling methods that increase sampling parallelism can leverage this available compute capacity without substantially adding to inference overhead.

Despite this potential, achieving efficient test-time scaling with mobile NPUs faces significant hardware challenges, which we categorize into two aspects:

Precision: Mobile NPUs were originally designed for coarse-grained quantized models and lack native support for fine-grained group quantization, which is essential for modern LLMs deployed in low bits. We observe that models quantized with conventional per-channel methods suffer severe performance degradation on reasoning tasks that are critical for test-time scaling.

Efficiency: While NPUs excel at matrix multiplication, their general-purpose vector units offer limited compute throughput and memory bandwidth. Many key non-matrix computations in LLM inference for test-time scaling must run on vector units, becoming a prominent bottleneck. Furthermore, the mismatch between the wide SIMD vector components and data granularity, coupled with the hardware’s memory access limitations, makes it difficult for software to fully utilize the computing power of vector units, further exacerbating the problem.

To address these challenges, we present an end-to-end LLM inference system that leverages the abundant compute

capacity of mobile NPUs to support test-time scaling workloads. To meet on-device resource constraints and precision requirements, we mainly adopt weight-only 4-bit fine-grained group quantization. For the resulting efficiency challenges, our solution incorporates the following key techniques:

Hardware-aware Tile Quantization Scheme: We present a novel matrix and vector unit-aware quantization layout. Through weight layout transformations before and after quantization, we apply fine-grained group quantization on hardware-friendly tiles and align with NPU’s memory access patterns, thereby minimizing runtime memory access overhead and maximizing vector compute utilization.

Efficient LUT-Based Computation: We replace complex key operations, including exponential computation in Softmax and the dequantization process in mixed precision GEMM, with efficient table lookup (LUT) instructions, alleviating computation bottleneck on the vector units.

We evaluate our system across three generations of Qualcomm Snapdragon platforms. Our proposed techniques bring up to $19.0\times$ speedup for mixed-precision GEMM and $2.2\times$ acceleration for Softmax compared to baselines, respectively. The results demonstrate the effectiveness of exploiting mobile NPUs for LLM test-time scaling workloads. Notably, we show that test-time scaling achieves state-of-the-art performance-cost trade-offs: using test-time scaling with smaller models can match or even surpass the performance of larger models running without scaling. To the best of our knowledge, this is the first work to explore the feasibility and evaluate the trade-offs of test-time scaling methods for LLMs with NPUs on mobile devices. Our contributions are summarized as follows:

- We analyze the architecture of modern mobile NPUs and identify underutilization of the specialized matrix units during the LLM decoding phases.
- We present two techniques: a hardware-aware tile quantization scheme and LUT-based computations to accelerate LLM test-time scaling on mobile NPUs.
- We design and implement an end-to-end LLM inference system² that leverages mobile NPUs to support test-time scaling workloads with minimal dependency on proprietary software stacks.
- We demonstrate that test-time scaling can effectively leverage otherwise wasted NPU compute capacity to enhance the generation quality of on-device small language models, achieving Pareto-frontier performance in accuracy and cost compared to traditional model scaling. It opens up new opportunities for deploying LLMs on mobile devices.

¹The value is obtained by dividing the 50 TOPS of the AMD Ryzen AI 9 HX 370 NPU by the 16 TOPS of the AMD Ryzen 7 8845HS NPU.

²Our code is available at <https://github.com/haozixu/llama.cpp-npu> (main repo) and <https://github.com/haozixu/http-ops-lib> (op library)

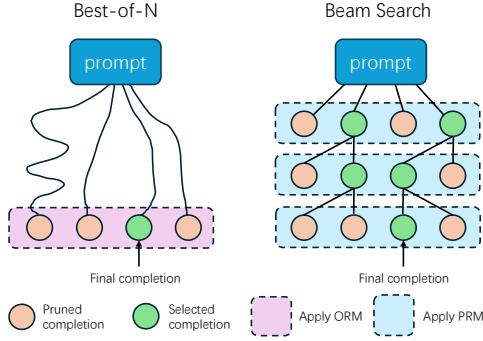


Figure 1. Two typical test-time scaling methods: Best-of-N and Beam Search.

2 Background

2.1 Scaling LLM Computation at Test-Time

Parallel test-time scaling emerges as a popular and effective new paradigm to improve model accuracy without modifying model parameters; instead, it devotes more computation at test-time. The simplest test-time scaling methods are majority-voting and self-consistency [3, 55], which are used to select the most consistent answer from multiple sets of generated samples. For math or programming problems with verifiable outcomes and domains with reward models (i.e., Outcome Reward Models), the highest scoring option can be chosen from completed sample sets, a strategy termed Best-of-N [50]. Through lookahead rollouts, methods similar to Monte Carlo Tree Search (MCTS) can select optimal paths from partially generated sequences, leading to the derivation of Process Reward Models (PRMs) [13, 43, 54, 67] that directly score intermediate results. With PRM assistance, lookahead-free step-level Beam Search [50, 57] dynamically discards low-quality generation paths to balance exploration and exploitation. Figure 1 illustrates the algorithm of two popular test-time scaling methods.

2.2 Neural Processing Units

With the growth of AI workloads, modern SoCs are increasingly integrating NPUs to accelerate neural network inference [9, 39, 48]. NPUs feature specialized acceleration of low-precision, computationally intensive core neural network operations (e.g., GEMM), delivering extremely high computational throughput while maintaining good power efficiency.

A widely adopted NPU architecture employs a "vector + matrix" combination, where the matrix unit accelerates operations like matrix multiplication and convolution, and the vector unit handles general-purpose computations such as normalization and complex activation functions. Well-known examples including Qualcomm's Hexagon NPU [39], Huawei's Ascend NPU [30], AMD's XDNA NPU [48], Intel NPU [9], and Intel's Gaudi HPU [10] all utilize this type

of architecture. Such NPUs differ significantly from common GPUs in their hardware execution model. As shown in Figure 2, in the GPU's SIMT model, different threads can independently perform branching, memory access, and computation, whereas in the NPU's SIMD-based execution model, a single thread operates on large vector or matrix data blocks. At the hardware level, NPUs typically employ fewer hardware threads and use VLIW architectures to reduce control logic overhead. Compared to GPUs, NPUs sacrifice programming flexibility and ease-of-use in exchange for higher execution efficiency and energy efficiency.

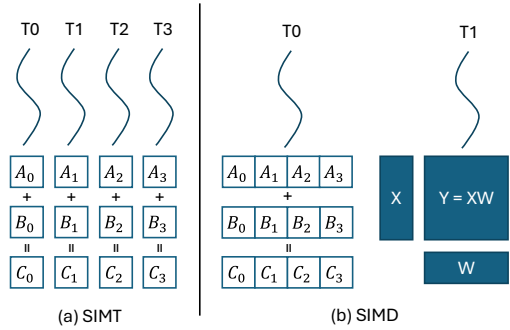


Figure 2. Comparison of (a) GPU's SIMT execution model and (b) NPU's SIMD execution model.

3 Motivation and Challenges

In this section, we first introduce some key features of mobile NPUs, and then analyze the opportunities of leveraging NPUs' free compute as well as the challenges in implementing efficient system for test-time scaling workloads.

3.1 Qualcomm's Hexagon NPU

The Hexagon NPU on Qualcomm's Snapdragon SoC is a representative mobile NPU due to its typical architecture, widespread adoption, and relatively accessible SDK. Therefore, we use it to demonstrate the core features of mobile NPUs.

3.1.1 Programming Interface. The primary approach to program Qualcomm's Hexagon NPU is through Qualcomm AI Engine Direct [47] (often referred to as QNN), a proprietary, closed-source DNN inference framework. In most cases, developers cannot customize high-performance low-level kernels even though the full LLVM toolchain for Hexagon NPU is provided in the Hexagon SDK, mainly because the instructions for the matrix unit remain undisclosed. We are able to utilize the FP16 matrix unit by reverse engineering the undocumented instructions in the binary libraries.

3.1.2 Architecture.

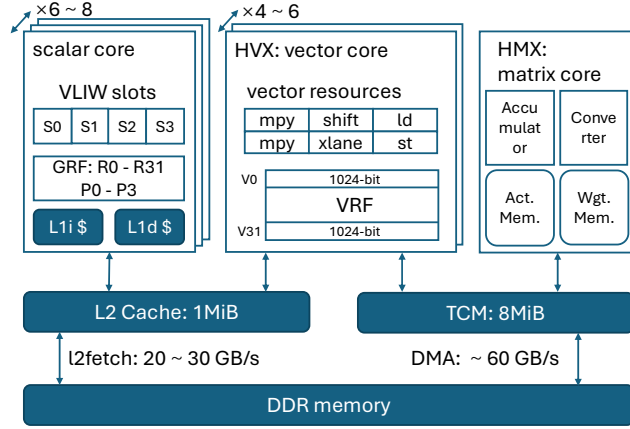


Figure 3. Hexagon NPU Architecture.

Computation Units. The Hexagon NPU features a typical hybrid architecture of “vector + matrix”. Its vector and matrix units are named HVX (Hexagon Vector eXtension) and HMX (Hexagon Matrix eXtension), respectively. The Hexagon NPU incorporates 6 to 8 scalar VLIW hardware threads for logical control. All vector or matrix instructions are issued from one of the four VLIW slots in a scalar core. The HVX unit context comprises 32 vector registers with a width of 1024 bits, and the number of such units ranges from 4 to 6. The number of HMX units is deduced to be 1 or 2.

Memory Subsystem. The Hexagon NPU includes a shared 1 MiB L2 cache and 8 MiB of TCM (Tightly Coupled Memory), the latter being a segment of software-managed on-chip memory. The HVX can read data from either the L2 cache or the TCM. Vector scatter/gather operations and all HMX instructions can only access TCM. Data can be loaded from DDR memory into the L2 cache and TCM via the 12fetch instruction and DMA mechanisms, respectively. Both support asynchronous transfers of 1D or 2D tensor data.

The HMX Unit. The powerful matrix multiplication capabilities of the Hexagon NPU originate from the HMX component. According to Qualcomm, the HMX unit supports various precisions, including INT4, INT8, INT16, and FP16 [39]. The following introduction is based mainly on FP16 HMX, with relevant information derived from reverse engineering, the Hexagon SDK, the QNN SDK, and publicly available information from Qualcomm.

The basic data unit for HMX operations is a tile, where each tile contains a matrix of a specific size. For FP16 HMX, a tile measures 32×32 , occupying 2 KiB of space. The HMX unit can load several tiles of weight memory and activation memory from the TCM. After performing matrix multiplication on each pair of matrix tiles, it accumulates the results into an internal accumulator. Finally, it outputs a tile corresponding to the accumulator. Meanwhile, the HMX unit can

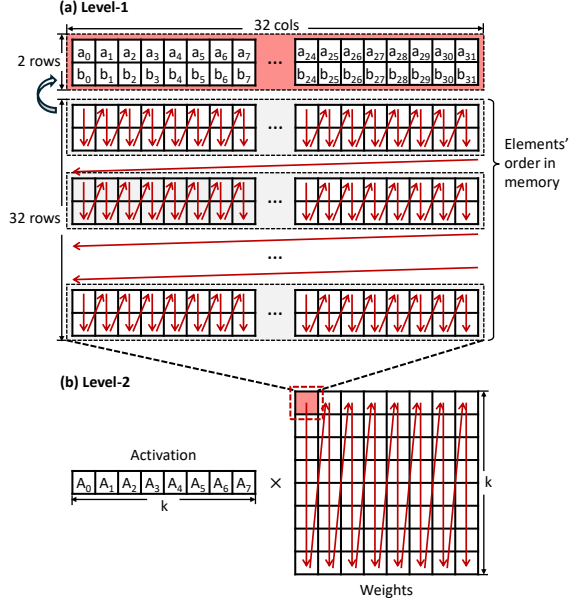


Figure 4. (a) The memory layout of FP16 HMX tile. Each tile corresponds to a 32×32 matrix and takes up 2048 bytes. Every two rows are permuted, having the same layout as the transposed 2×32 sub-matrix. (b) The overall memory layout for HMX-based GEMM. The weight tiles are arranged in column-major layout since the hardware performs inner-product at tile level.

independently scale and add biases to each channel (column) of the output tile.

FP16 HMX tiles have a special memory layout, as shown in Figure 4 (a). Both input and output tiles follow this layout. A typical way to construct this layout is to use HVX instructions to perform cross-lane shuffling on every two adjacent rows of the original matrix.

3.2 Opportunities: Free Matrix Computation During LLM Decoding

During the autoregressive generation process, LLM’s input typically corresponds to only one token, which causes the GEMM operation to degenerate into GEMV. For example, an activation matrix of shape $[1, \text{hidden_dim}]$ is multiplied by a weight matrix of shape $[\text{hidden_dim}, \text{proj_dim}]$. In the case of using FP16 HMX, the effective size of each compute tile is $[1, 32] \times [32, 32]$. Since the basic unit of hardware computation is a 32×32 tile, 31 rows in the input activation tile do not correspond to actually useful content, resulting in low utilization of the matrix unit and waste of computing power.

Meanwhile, some test-time scaling algorithms can achieve better generation quality by increasing the computation during generation, including parallel sampling methods such as

Self-Consistency, Best-of-N and Beam Search. Their characteristic is that they explore multiple generation paths using a batch size greater than 1 and use certain ways (e.g., an external verifier) to select the better generation paths. Figure 5 shows an example of test-time scaling using Best-of-N. As the generation budget (i.e. the maximum batch size in the decoding phase) increases, the model accuracy on the MATH500 dataset improves significantly.

Based on these, we propose running the test-time scaling workloads of LLM on mobile NPUs. In this way, the computing power of the NPU wasted during the conventional LLM generation process can be effectively leveraged. The decoding overhead will not increase significantly in theory, and the generation quality of the model can be improved at run-time without modifying the model weights.

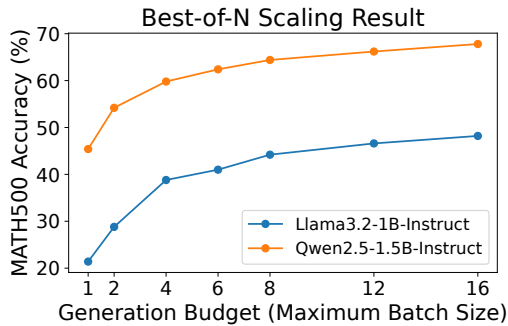


Figure 5. An example of test-time scaling with two models. The accuracy on MATH500 improves as generation budget increases.

3.3 Challenges

Although it is theoretically feasible to utilize mobile NPUs for test-time scaling, an efficient implementation faces numerous hardware challenges. We summarize these challenges as follows.

Insufficient Precision. Although HMX units support FP16 GEMM, deploying FP16 models on resource-constrained devices remains impractical, making quantized models the typical alternative. The matrix units in most mobile NPUs, including the HMX, were originally designed to accelerate integer-quantized DNN models that employ coarse-grained quantization schemes, such as per-tensor or per-channel quantization. As a concrete example, Hexagon NPUs lack native hardware support for the fine-grained quantization methods essential to modern LLMs. This limitation is further reflected in the software stack: QNN only supports per-tensor or per-channel weight quantization. Applying coarse-grained low-bit quantization directly to LLM weights can lead to significant accuracy degradation.

As shown in Table 1, the accuracy results of the Llama 3.2 1B-Instruct model under QNN’s per-channel quantization³ and AWQ per-group 4-bit quantization (both under W4A16 settings) indicate that the per-channel quantized model suffers severe performance degradation in challenging mathematical reasoning tasks. Unfortunately, since test-time scaling methods are applied in such tasks, the baseline accuracy achieved by QNN fails to meet even the minimal requirements for performance scaling.

dataset	AutoAWQ (W4A16)	QNN (W4A16)
MATH500 (↑)	15.9	2.1
GSM8K (↑)	32.6	3.4
Wiki PPL (↓)	19.42	28.99

Table 1. Comparison of Llama3.2-1B-Instruct’s performance under different implementation. QNN’s quantization drastically hurts model’s reasoning ability.

Weak General Purpose Compute and Memory Bandwidth. In the absence of native hardware support for fine-grained group quantization, a common approach is to rely on general-purpose computing units to handle such computations. However, we discover a significant gap between the compute and memory access capabilities of the general-purpose vector units and the specialized matrix units within the NPU. We measure the FP16 GEMM performance of both the HVX and the HMX on the Hexagon V75 NPU using a $1024 \times 1024 \times 1024$ GEMM operation, with all inputs and outputs residing in on-chip TCM to reflect the hardware’s peak performance. As shown in Table 2, the FP16 GEMM throughput of the matrix unit reaches up to 12 TFLOPS — over 300 times higher than that of a single vector thread. In terms of memory bandwidth, a dedicated DMA engine achieves over 60 GB/s read bandwidth from DDR, whereas the vector unit’s memory read bandwidth via the core data path remains below 30 GB/s. However, the high bandwidth provided by DMA is restricted to large, regular 1D or 2D data blocks and cannot efficiently handle small or irregular memory accesses. These observations highlight that the general-purpose compute and memory bandwidth of the vector unit are insufficient to keep up with the computational throughput of the specialized matrix unit, posing a major challenge for implementing high-performance mixed-precision GEMM kernels under fine-grained quantization.

4 Design Overview

We present an LLM inference system designed for mobile NPUs and optimized for test-time scaling workloads.

To address the accuracy challenge, we adopt 4-bit fine-grained group quantization for the primary weights while

³We use the official model released by PowerServe, available at <https://huggingface.co/PowerServe/Llama-3.2-1B-PowerServe-QNN29-8G3>

hardware units	HVX (1 Thread)	HMX
FP16 GEMM GFLOPs	32.93	12032.54
memory read bw. (GB/s)	26	60 (DMA)

Table 2. The performance metrics of the HVX and HMX units, in terms of FP16 GEMM computing power (in GFLOPs) and memory read bandwidth (in GB/s).

keeping activations in floating-point. During runtime, we dynamically dequantize the weights into floating-point values on the fly, leveraging the powerful FP16 matrix computation capabilities of the NPU to efficiently support test-time scaling tasks.

For the unavoidable general-purpose computations — where the vector processing units exhibit limited memory bandwidth and compute throughput, our core strategy includes:

- Employing hardware-aware offline design to minimize runtime computation overhead;
- Fully exploiting the intrinsic capabilities of SIMD vector units to bridge the gap between specialized hardware and flexible software requirements.

Specifically, we introduce the following techniques:

Hardware-aware Fine-grained Tile Quantization Scheme.

We propose a novel quantization layout that performs group quantization in fine-grained rectangular tiles, as opposed to conventional approaches that group along the accumulation axis. To align with the memory access patterns of both the matrix and vector units, we introduce an offline pipeline involving weight pre-quantization transformation, quantization, and post-quantization transformation. This enhances the continuity of runtime memory access and eliminates unnecessary computational overhead.

Efficient LUT-Based Computation. For more complex runtime operations, we leverage the vector unit’s lookup table (LUT) instructions and generalized LUT mechanisms to replace intricate transformation logic. This approach accelerates key bottleneck operations in test-time scaling workloads, including dequantization within mixed-precision GEMM and the Softmax operation in Attention.

5 System Design

5.1 Hardware-aware Fine-grained Tile Quantization Scheme

Existing work [28, 34] has shown that quantization errors significantly degrade model performance in challenging tasks like mathematical reasoning. However, due to stringent on-device resource constraints, full-precision models remain infeasible, making fine-grained quantization essential to maintain accuracy.

Unfortunately, implementing efficient dequantization-based GEMM kernels under fine-grained quantization on mobile

NPUs introduces substantial system challenges. We identify two primary issues:

- mismatch between the weight layout expected by the matrix unit and conventional group quantization layout;
- suboptimal utilization of the wide vector registers caused by small group sizes.

To overcome these limitations, we propose a novel tile quantization scheme that incorporates two components:

- a tile-based quantization strategy designed to align with the matrix unit’s inherent data layout;
- a post-quantization weight permutation method that maximizes utilization of the vector unit’s processing capabilities.

5.1.1 Tile-Group Quantization. In conventional quantized GEMM, weight matrices are typically stored in column-major layout, which aligns with the vector dot-product operations used in CPU-based matrix multiplication, such as in the llama.cpp CPU backend. The weights are divided into contiguous quantization groups — typically of size 32 — along the column dimension. Within each group, the values are quantized, and the resulting integer weights, along with their corresponding scale and zero-point parameters, are stored interleaved in memory, preserving the original column-wise ordering of the matrix.

However, on NPUs with special matrix units, the conventional group layout is often misaligned with hardware requirements. As illustrated in Figure 6, elements that are contiguous in the conventional layout become scattered in on-chip TCM. For SIMD vector units, such non-sequential access patterns are problematic. Although modern vector engines provide gather/scatter operations to alleviate scattered accesses, these operations remain expensive. Simply transposing the weight matrix does not resolve the mismatch, as the complex multi-level data layout expected by the matrix unit still results in noncontiguous memory access.

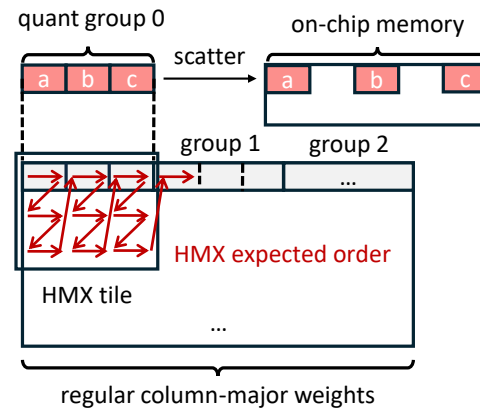


Figure 6. A simplified illustration of the mismatch between the quantization group layout and HMX tile layout.

To address this, we first permute the weights into the layout expected by the matrix unit, and then apply round-to-nearest quantization group by group. For a group size of 32, this method effectively performs group quantization in units of 2×16 tiles. Given that pretrained weights in typical models approximately follow a zero-mean Gaussian distribution, quantizing within these reshaped tile groups does not significantly alter the statistical properties within each group compared to conventional grouping. Therefore, the resulting quantization error remains comparable.

Specifically, we arrange the weights before quantization according to the layout shown in Figure 4, which is hierarchically structured into two levels: an outer column-major ordering of tiles, matching the tile-level inner product operation of the matrix unit, and an inner shuffling of every two rows within each tile. We then quantize the weights group-wise in the new memory order.

5.1.2 Coalescing Quantization Groups for Wide Vector Accesses. By default, quantized weights are stored in an Array of Structures (AoS) layout. Taking Q4_0 symmetric quantization as an example, each group of 32 elements consists of 16 bytes of INT4 quantized values and 2 bytes of FP16 scale values, with quantized values and scales interleaved in memory. Since memory access on the NPU architecture relies heavily on software-managed local 1D or 2D prefetching, we avoid the Structure of Arrays (SoA) layout, where quantized values and scales reside in separate large contiguous arrays, to better align with the hardware’s preferred access pattern.

However, fine-grained quantization groups introduce a mismatch with the native vector processing granularity: A single quantization group is too small to fill a 128-byte wide vector register. Accessing such small groups would require multiple memory operations or additional instructions to merge data from multiple registers, resulting in inefficient memory bandwidth usage and computational overhead.

To solve the issue, we coalesce 8 quantization groups into a larger super-group and reorganize its content such that the INT4 values from 256 consecutive elements occupy exactly one full HVX register. This process is illustrated in Figure 7.

5.2 LUT-Based Computations

Given the limited general-purpose computing performance of the vector unit, we propose using generalized look-up table (LUT) instructions to replace complex computations, thereby reducing instruction count and computational overhead. LUT-based computation is particularly effective for accelerating key operations in test-time scaling workloads, such as the exponential function in Softmax and the dequantization process.

5.2.1 Fast Softmax via Vector Gather. Test-time scaling methods typically increase sampling parallelism, leading to larger batch sizes and longer context lengths. We analyze

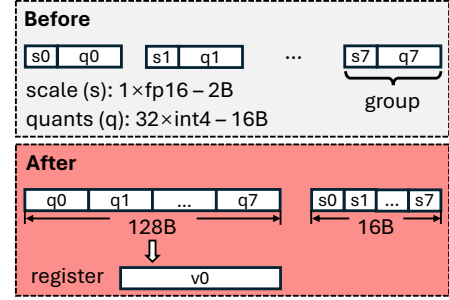


Figure 7. Repacking 8 fined-grained quantization groups into a super-block. The INT4 quantized values fit in a vector register.

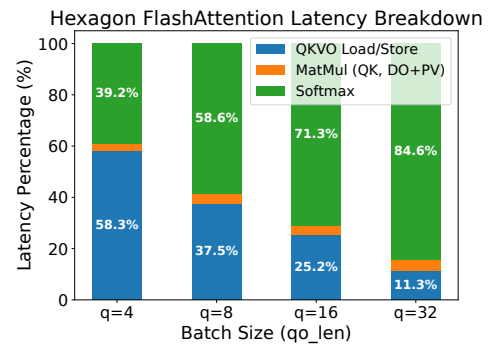


Figure 8. FlashAttention latency breakdown on Hexagon NPU. We use Qwen2.5-1.5B and prompt length is set to 4096.

the impact of these scaling factors on the major operators in transformer-based LLMs during generation:

- **GEMM.** Based on previously described NPU hardware characteristics, moderately increasing batch size in test-time scaling workloads does not substantially increase GEMM latency. Moreover, GEMM latency is independent of context length.
- **Misc. Ops.** For operators such as activation functions, LayerNorm, residual Add, and RoPE, although their computational overhead is roughly proportional to input size, we neglect their impacts due to their small computation and memory access volumes.
- **Attention.** The theoretical computational complexity of Attention scales with both batch size and context length, making it a potential performance bottleneck in test-time scaling scenarios.

We implement FlashAttention [11] on the Hexagon NPU using FP16 HMX and measure its latency composition at a prompt length of 4096 under various input batch sizes (query lengths), as shown in Figure 8. The results indicate that matrix multiplication contributes little to overall latency, whereas Softmax dominates Attention execution time as the query length increases.

Our analysis shows that the primary bottleneck of on-chip Softmax lies in the exponential computation, which must be applied to $\Theta(N_q \times N_{kv})$ elements. Adding to the issue, these expensive exponential operations must be executed on the HVX, which lacks dedicated hardware support for special math functions. Following common practice, we replace $\exp$ with $\exp_2$ and absorb the coefficient $\log_2 e$ in the QK^T scaling factor $\frac{1}{\sqrt{d}}$. For an input element x decomposed into integer part k and fractional part f , 2^f is approximated using a Taylor series polynomial expansion, while k is directly added to the exponent field of 2^f 's IEEE-754 representation. However, polynomial evaluation involves sequential dependencies, limiting instruction-level parallelism under the VLIW architecture.

To alleviate the exponential computation bottleneck, we explore replacing explicit exponent calculation with a pre-computed lookup table (LUT). The HVX provides the `vgather` instruction, which can gather values from scattered locations in the TCM into a contiguous 128 byte TCM region. Although `vgather` can implement large LUTs, using LUTs for $\exp$ remains challenging: storing 2^{32} elements for 32-bit floats is impractical. Furthermore, `vgather` itself introduces substantial latency — 24 to 48 instruction packets on Hexagon V75, so its usage must be minimized.

To enable practical LUT-based $\exp$, we design the following approach. First, we extensively use FP16 throughout FlashAttention, with the on-chip computation process outlined in Algorithm 1. The matrices S, P, O and the vectors $\tilde{m}, \tilde{l}$ are stored in 16-bit floats, with both the input and output of the $\exp$ computation in 16-bit floats. In particular, FP16 HMX uses higher-precision floating-point numbers for accumulation internally, and we upcast elements to 32-bit precision for critical operations such as row-wise summation of matrix P .

Using 16-bit inputs and outputs restricts the LUT to 65536 entries, requiring 128 KiB of storage, which fits within the TCM. A variant of `vgather` supports gathering 64 2-byte elements in one instruction, with a maximum address offset of 65536 bytes. However, 65536 FP16 entries occupy 128 KiB, leaving half of the entries inaccessible with direct addressing. To solve this, we leverage the property of safe softmax [42], which ensures that all inputs to $\exp$ are non-positive by subtracting the row-wise maximum m_i . Thus, we only store values for $x \leq 0$, resulting in a LUT with 32768 entries (64 KiB). During LUT-based $\exp$ computation, we ignore the MSB (sign bit) of the FP16 input and left-shift the input by one bit to generate the byte offset required by `vgather`.

The LUT is precomputed during system initialization, introducing no additional overhead during model inference. It occupies a fixed 64 KiB region in TCM, accounting for only $64\text{KiB}/8\text{MiB} \approx 0.8\%$ of the total TCM capacity, thus minimally impacting TCM availability for other operations.

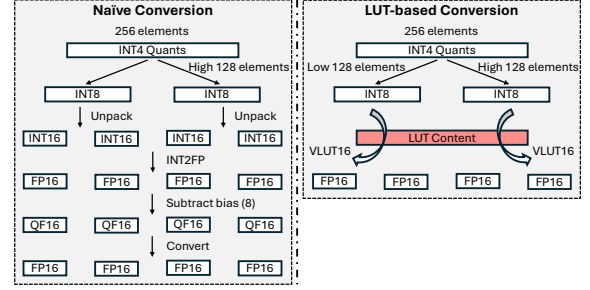


Figure 9. Converting INT4 quantized values into FP16 numbers via table lookup.

5.2.2 LUT-Centric Efficient Dequantization. The runtime HVX dequantization requires careful design to avoid additional overhead. We present an efficient dequantization process based on the HVX lookup table instructions. The `vLUT16` instruction is capable of performing a table lookup in a table of 16 elements for each 8 bit index in a source vector register. Each input byte is transformed into a 16-bit value, therefore `vLUT16` results in a pair of registers.

Fast INT4 to FP16 conversion via table lookup. Using `vLUT16` instructions, we directly transform 4-bits quantized values into $[-8, 7]$ FP16 values for Q4_0 quantization scheme, avoiding the conventional mask-unpack-convert instruction sequence. Figure 9 demonstrates the comparison of two approaches. For Hexagon NPU prior to V79, all HVX floating-point operations produce results in an internal format called `qfloat`, which requires extra instructions to convert back to standard IEEE-754 formats. The use of table-lookup eliminates these overheads. This LUT-centric design can easily support different 4-bit encoding schemes (e.g. FP4, NF4 [12], IQ4_NL used in llama.cpp) simply by adjusting the table contents.

Scales broadcast via table lookup. A 128-byte HVX register can accommodate two FP16 quantization groups of size 32. Therefore, the conventional approach is to broadcast scalar scales to the entire vector register and then concatenate two registers for subsequent multiplication with quantized values. However, by using the scales of four groups as LUT contents and applying predefined constant indices, we can achieve the broadcast of four groups of scales with just one `vLUT16` instruction.

6 Implementation

Our inference system is implemented on top of llama.cpp [15] with approximately 7K lines of code in C/C++ and inline assembly. We use the LLVM toolchain in the Hexagon SDK (version 6.0.0.2) to generate code for Hexagon NPUs. We especially note that our system has no dependency on Qualcomm's QNN, avoiding inflexible static fixed-shape computation graphs.

Algorithm 1: On-chip computation of ours FP16 FlashAttention (different heads omitted)**Input:** Head dimension d , Number of Query tiles T_q , Number of KV tiles T_{kv} , Query tile size B_q , KV tile size B_{kv} **Input:** Matrices Q_i (FP16) $\in \mathbb{R}^{B_q \times d}$, K_j, V_j (FP16) $\in \mathbb{R}^{B_{kv} \times d}$

- 1 Initialize $O_i^{(0)} = (\mathbf{0}) \in \mathbb{R}^{B_q \times B_{kv}}$ (FP16), $m = (-\infty) \in \mathbb{R}^{B_q}$ (FP16), $l = (\mathbf{0}) \in \mathbb{R}^{B_q}$ (FP16);
- 2 $S_i^{(j)} = \text{MatMul}(Q_i, K_j^T, \text{AccumType}=\text{FP32}) \in \mathbb{R}^{B_q \times B_{kv}}$ (FP16);
- 3 $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(S_i^{(j)})) \in \mathbb{R}^{B_q}$ (FP16);
- 4 $P_i^{(j)} = \text{LUT_Exp}(S_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_q \times B_{kv}}$ (FP16);
- 5 $l_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} l_i^{(j-1)} + \text{rowsum}(P_i^{(j)}, \text{AccumType}=\text{FP32}) \in \mathbb{R}^{B_q}$ (FP16);
- 6 $O_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}}) O_i^{(j-1)} + P_i^{(j)} V_j \in \mathbb{R}^{B_q \times d}$ (FP16, AccumType=FP32);

Output: $O_i = \text{diag}(l_i^{(T_{kv})})^{-1} O_i^{(T_{kv})}$

Our implementation mainly consists of two modules: one module is the operator library for the Hexagon NPU, which is compiled into an independent Hexagon DSP shared object; the other module is integrated with llama.cpp on the CPU side. The NPU operator library implements computation kernels, power management, hardware resource management, and a computation thread pool. We add a Hexagon NPU backend to llama.cpp, leveraging rpcmem shared memory as the underlying buffer type. rpcmem is a wrapper for the kernel dmabuf memory and supports the sharing of physical memory between the CPU and the NPU. The related allocation, deallocation, and mapping interfaces are provided by libcdsprpc.so in the Android system's vendor libraries. By utilizing shared memory buffers, we not only eliminate unnecessary inter-processor data copy but also reuse the existing memory management system as much as possible. In addition, we are able to schedule the operators that have not been implemented on the NPU to run on the CPU, achieving seamless integrations with upper-layer applications.

During the backend initialization phase, we call the FastRPC [46] facility of the Hexagon SDK to start the remote NPU session and initialize an area of shared memory for communication. On the NPU side, a thread continuously polls in this shared-memory area to receive computation requests from the CPU. Compared to the default RPC implementation, communication through shared memory can have a lower latency. We note that after the CPU writes data to the shared memory, the NPU will not automatically invalidate the cache of the corresponding area as there is only one-way coherence between the CPU and the NPU on the Snapdragon SoC. Therefore, we manually clear the cache before NPU polls. Similar cache maintenance operations are also required for shared buffers containing model activations.

7 Evaluation

7.1 Experiment Setup

Devices. The experiments on NPU performance are conducted on three Android devices: OnePlus Ace3, OnePlus 12,

Device	SoC	NPU Arch.
OnePlus Ace3	Snapdragon 8 Gen 2	V73
OnePlus 12	Snapdragon 8 Gen 3	V75
OnePlus Ace5 Pro	Snapdragon 8 Elite	V79

Table 3. Mobile devices used in evaluation.

OnePlus Ace5 Pro. Some of the accuracy results are obtained on a server testbed equipped with NVIDIA RTX3090 GPUs.

Models. We choose models from the Qwen 2.5 [63] and Llama 3.2 [40] model family. Considering the actual resource limitations of mobile phones, we mainly evaluate Qwen 2.5 with model sizes of 1.5B and 3B, as well as Llama 3.2 with model sizes of 1B and 3B, which correspond to practical deployable model sizes. When evaluating the performance-cost trade-off of time-time scaling methods, we additionally consider Qwen 2.5 with a model size of 7B. In the evaluation of mathematical reasoning tasks, we use the Instruct model variants of Qwen 2.5 and Llama 3.2. For Best-of-N search and step-level beam search, Skywork-1.5B-PRM [43] is used as the outcome-reward and process-reward scorer.

Datasets and metrics. In the test-time scaling tasks, we evaluate the pass@1 accuracy of the models in two mathematical reasoning tasks, MATH500 [20] and GSM8K [8], and we uniformly use the 0-shot CoT prompt. For other accuracy measurements, the WinoGrande [49] accuracy, the MMLU [19] accuracy, and the Wikitext-2 perplexity are evaluated using llama-perplexity utility.

Baselines. Since we focus on test-time scaling tasks, we mainly present the performance of our implementation under different decoding workloads. To demonstrate the advantages of using NPUs to run test-time scaling workloads, we select the recent OpenCL backend of llama.cpp⁴ as the GPU-based system for comparison. This OpenCL backend incorporates optimized Q4_0 matrix multiplication kernels tailored for Snapdragon's Adreno GPU. Since existing NPU-based systems all have certain limitations in handling test-time scaling workloads, we do not use them as the primary

⁴commit: 1caae7f

baselines: `llm.npu` [59] does not utilize the NPU for computation during the decoding phase; other QNN-based systems have low accuracy (e.g., PowerServe [2]); and systems like Powerinfer-2 [61] and HeteroLLM [6] are not open-source. Nevertheless, we still report the QNN-based data as a reference in Section 7.2.4.

Settings. In the operator-level evaluation of GEMM, we select the sizes of the weight matrices of the linear layers corresponding to Qwen2.5-1.5B, Qwen2.5-3B, Llama3.2-1B, and Llama3.2-3B. Specifically, these include the Attention projection matrices W_q, W_o and $W_{gate}, W_{up}, W_{down}$ in the Feed Forward Network (FFN). (For modern models that use Grouped Query Attention (GQA), the projection matrices W_k, W_v in Attention are not selected because their scale is smaller compared to W_q, W_o). Most of the matrices adopt the Q4_0 quantization scheme, which corresponds to 4.5 Bits Per Weight (BPW). As for the FFN down matrices, we apply the Q8_0 quantization scheme (8.5 BPW) to reduce quantization errors, as existing work indicates their importance in preserving model accuracy [26, 31, 33].

7.2 Overall Performance

7.2.1 Accuracy-Latency Trade-off of Test-time Scaling. Figure 10 illustrates the performance-cost trade-off of the test-time scaling methods. We use the accuracy in MATH500 and GSM8K as metrics for generation quality and the average decoding latency of on-device models as the cost metric (the data here account for the increased context length introduced by TTS). In the figure, the top row and the bottom row correspond to Best-of-N and Beam Search results, respectively, while "QN"/"LN" denotes the Qwen2.5 or Llama3.2 models with N billion parameters. The SoC results exclude the "8G2" entry due to a known NPU virtual address space limitation [17] of Snapdragon 8 Gen 2 that prevents models with 3B or more parameters from running. The isolated points marked with a "base" represent the average performance obtained via conventional sampling with the models.

The data show that test-time scaling offers a trade-off space and achieves a more superior Pareto frontier under specific configurations, enabling a better performance-cost balance. In the Best-of-N method, the scaling results of Qwen2.5 1.5B and 3B outperform the baseline accuracies of the 3B and 7B models, respectively. For Beam Search, Qwen2.5-1.5B and Llama3.2-1B can achieve efficiency comparable to or slightly better than their respective 3B variants. Our results indicate that by leveraging the computing power of NPUs and test-time scaling algorithms, small on-device models have the potential to surpass larger models in terms of both generation quality and inference cost.

7.2.2 On-Device Decoding Performance. Figure 11 demonstrates the on-device decoding throughput of different models in different batch sizes. We only evaluate Qwen2.5-1.5B

and Llama-3.2-1B on OnePlus Ace3 due to a 2GiB limitation of the virtual address space on older NPUs.

The data show that for the three devices, the end-to-end decoding throughput of the system significantly increases as the batch size increases. The fundamental reason for the increase in decoding throughput is that the idle computing power of the HMX unit is utilized, and, essentially, the computation time consumed on the core HMX does not increase at all. However, the decoding throughput does not scale perfectly linearly because the inference process contains parts that become much slower with the growth of the input length. Specifically, in our implementation, we conservatively place the weights of the `lm_head` (the projection matrix from the hidden states to the vocabulary) and the related activations on the CPU instead of the NPU. Modern LLMs have a large vocabulary, making the `lm_head` and logits occupy a large space. Unfortunately, the Hexagon NPU only has a 32-bit virtual address space, therefore placing the complete logits tensor on the NPU may prevent the complete model from running. Currently, we observe that when the batch size equals 16, the proportion of the computation time of logits on the CPU is close to or exceeds 50%. We expect that after addressing the limitations of the NPU address space and placing the logits computation on the NPU, the system will achieve better throughput scaling characteristics.

7.2.3 Power and Energy Consumption. We measure the power consumption during LLM decoding via sysfs interface on OnePlus 12 with the performance mode enabled. As the batch size increases in the decoding phase, the power consumption of running the 1.5B Qwen model increases, but the overall power consumption of the device is still within 5W; in contrast, the power consumption corresponding to running the 3B Qwen model stabilizes at around 4.3W. Figure 12 shows the normalized energy consumption, which is calculated by multiplying the corresponding power consumption by the relative decoding latency. The scaling trait of energy consumption with respect to the batch size is similar to that of decoding latency; therefore, replacing the cost metric in Figure 10 with energy also results in similar accuracy-cost trade-off characteristics. In particular, we note that the decoding energy consumption of the 1.5B model at a batch size of 8 is lower than that of the 3B model at a batch size of 1, while the test-time scaling accuracy of the 1.5B model when decoding with a batch size of 8 on mathematical tasks is comparable to the base accuracy of the 3B model.

7.2.4 Comparison with Other Systems. The decoding and prefilling performance of our system is presented in Figure 13. We compare our system against a GPU-based implementation and add the performance of FP16 QNN as a reference. During the decoding phase, although the GPU decodes faster at batch size 1, our NPU-based system exhibits higher decoding throughput and better scaling characteristics at larger batch sizes, highlighting the advantage of

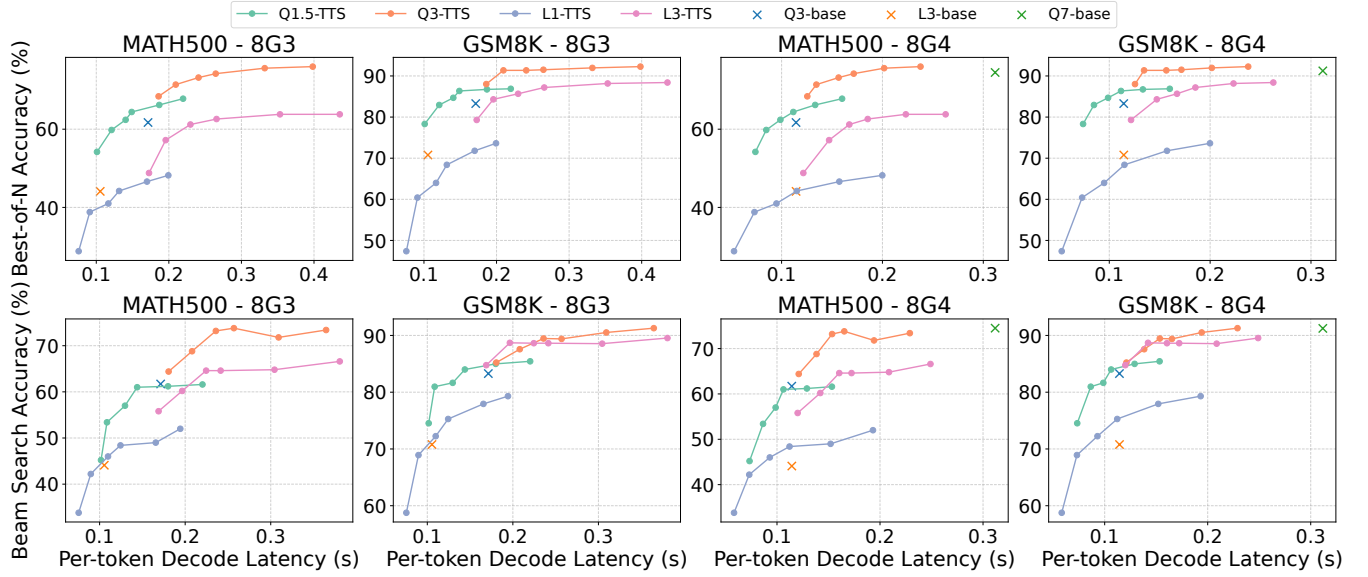


Figure 10. Accuracy-latency trade-off of different test-time scaling methods on various combinations of dataset and hardware.

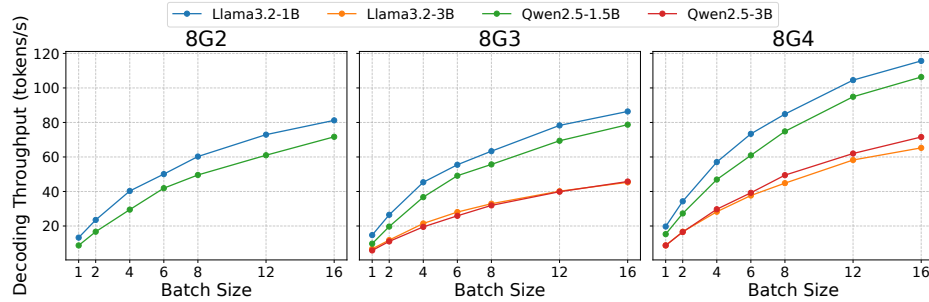


Figure 11. End-to-End decoding throughput of different models under various batch sizes and hardware settings.

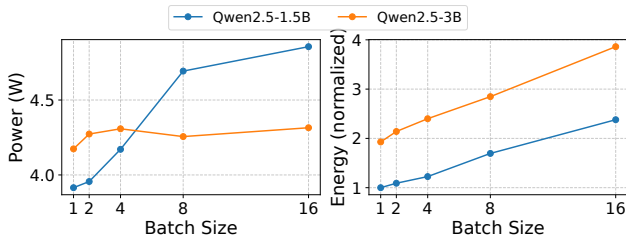


Figure 12. Power and energy consumption during the LLM decoding stage.

using NPUs in test-time scaling workloads. Our system also consistently outperforms the GPU-based system in terms of prefiling throughput, achieving comparable performance with proprietary QNN under certain workloads.

dataset	Tile group	Common group	F16
WinoGrande ($\uparrow$)	62.559	63.349	64.613
MMLU ($\uparrow$)	35.465	35.271	34.819
Wiki PPL ($\downarrow$)	10.206	10.190	9.798

Table 4. Accuracy comparison between models using tile quantization groups tailored for HMX layout and models using conventional quantization groups.

7.3 Accuracy Assessment

Quantization Scheme. We evaluate the accuracies of the Qwen2.5-1.5B model corresponding to the tile quantization groups based on the HMX layout and the conventional quantization groups. As shown in Table 4, the model using our quantization layout has slightly higher accuracy in MMLU compared to the model with the conventional layout, and there is only a slight decrease in Winogrande and Wikitext PPL. Moreover, these accuracy differences are much smaller than the performance loss caused by quantization itself (as

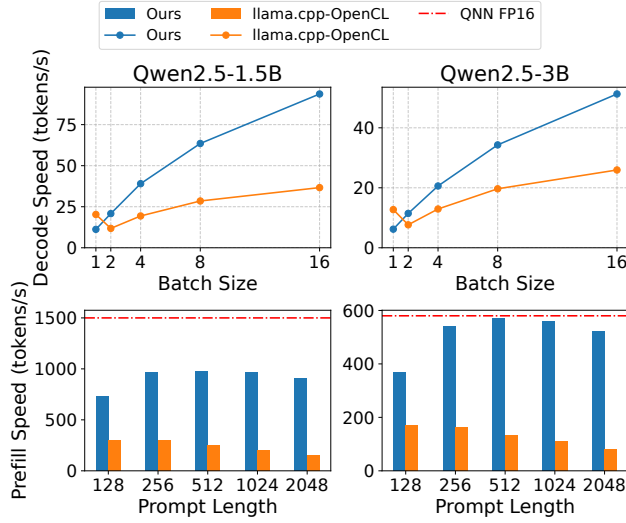


Figure 13. Inference throughput comparison.

indicated by Wikitext perplexity in the "F16" column). In general, using our proposed tile quantization group does not lead to a significant decrease in the accuracy of the quantized model.

dataset	Our LUT16 FA	F32 Attention
WinoGrande ($\uparrow$)	62.796	62.559
MMLU ($\uparrow$)	35.207	35.465
Wiki PPL ($\downarrow$)	10.205	10.206

Table 5. Accuracy comparison between models using our F16 FlashAttention with LUT-based Softmax and models using conventional F32 Attention.

Attention Implementation. Table 5 shows the model accuracies corresponding to our LUT-based FP16 Attention and the conventional FP32 Attention, using the same model and datasets as above. It can be seen that replacing the non-critical parts in Attention (except for the accumulation) with a lower FP16 precision does not have a noticeable impact on the end-to-end accuracy of the model.

7.4 Ablation Study

Softmax in Attention. Figure 14 shows the on-chip softmax latency corresponding to the calculation of the exponential function exp using different methods under different attention workloads. The length of the input query for Attention is set to 1, 4, and 16, while the length of KV is set to 1024, 4096, or 16384. The figure indicates that our LUT-based exponential calculation achieves an acceleration of 1.26 to 2.19 times compared to the conventional 32-bit floating-point exp, and up to 1.60 $\times$ speedup compared to the 16-bit floating-point exp. It is worth noting that when pre-computing the exp lookup table, floating-point numbers with a width of

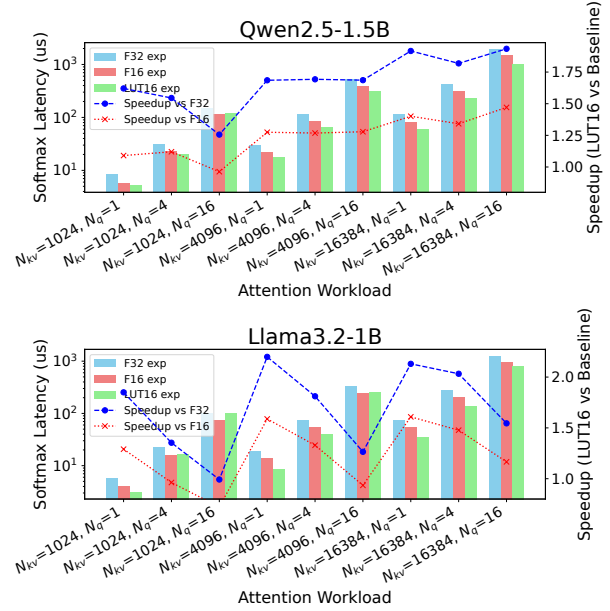


Figure 14. Ablation study of on-chip softmax of our proposed F16 Attention with LUT-based exponential computation. Performance is measured on OnePlus 12.

32 bits or higher can be used to calculate the intermediate results. Therefore, the LUT-based exp has a higher accuracy than the 16-bit polynomial approximation of exp. When the context length is short, a larger input query will slightly reduce the acceleration ratio, but this phenomenon will be alleviated when the KV length is longer.

Dequantization-based GEMM. Figure 15 presents the ablation experiment for optimization of the GEMM dequantization layout. The baseline method corresponds to the conventional memory layout, where the column-major weight matrix is quantized according to the continuous groups in memory. The GEMM kernel dequantizes the 32-sized groups one by one during runtime and then scatters the elements to the correct positions in the TCM. The item of "HMX layout" applies the offline weight rearrangement and tile quantization group for the HMX layout, enabling the FP16 weights to be continuously written into the TCM. "Ours" is the version that adopts all the optimizations including the quantization group coalesce. In addition, we add a set of data labeled "no dequantization". In this implementation, instead of performing actual weight dequantization, the quantized weights are read directly from the memory and copied to the on-chip memory without any computation. This set of data can be regarded as the performance upper bound of dequantization-based methods.

Compared to the baseline, our method achieves an acceleration of 9.65 to 19.04 times under different matrix sizes. This is mainly because the scatter operations in the baseline

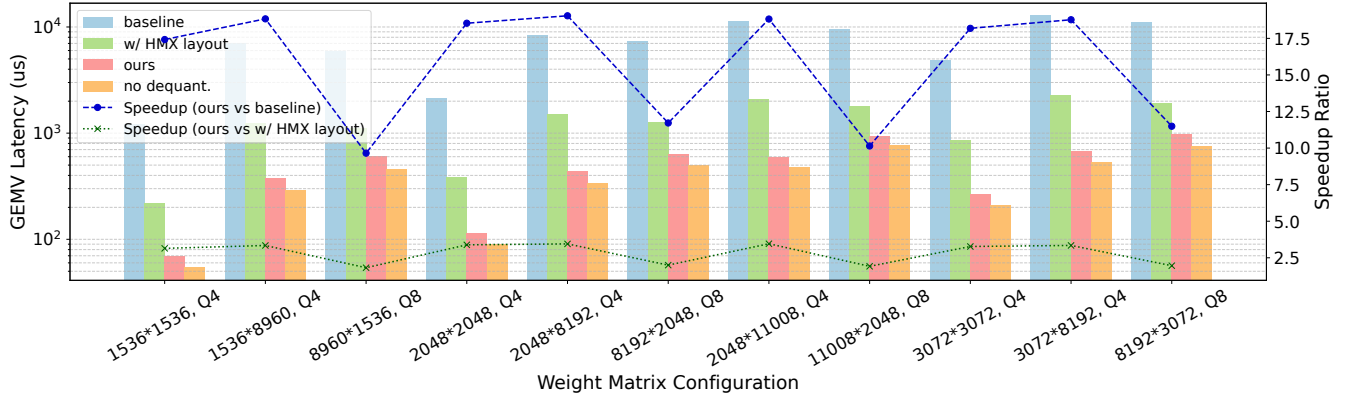


Figure 15. Ablation study of proposed optimizations on GEMM dequantization. We measure the performance of GEMV on OnePlus 12.

are extremely costly. After applying the HMX layout, the quantization group coalesces and the rearrangements also effectively reduce computational waste, bringing a speedup of 1.82× to 3.45×. In particular, compared to the "no dequantization" group, our method is only 27% slower on average, indicating that this implementation is already close to the performance upper limit of dequantization.

7.5 Overhead and Sensitivity Analysis

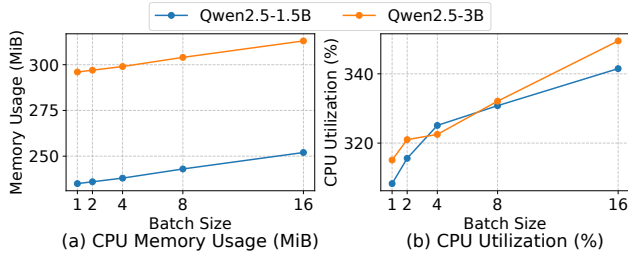


Figure 16. CPU and memory usage during the decoding stage.

CPU and Memory Usage. We evaluate the CPU utilization and memory consumption of the 1.5B and 3B Qwen2.5 models during the decoding stage on OnePlus 12. The CPU memory usage presented in Figure 16 is derived from the resident memory size reported by the top command. We also measure the total size of dmabuf (i.e., memory used by NPU) using pmap, yielding constant values of 1056 MiB and 2090 MiB under a context budget of 4096 tokens for the 1.5B and 3B models, respectively. The total memory consumption is approximately 1.3 GiB for the 1.5B model and 2.4 GiB for the 3B model. The CPU utilization increases with batch size due to the increased computation of vocabulary projection on CPU, yet the number of utilized cores is consistently limited to 4.

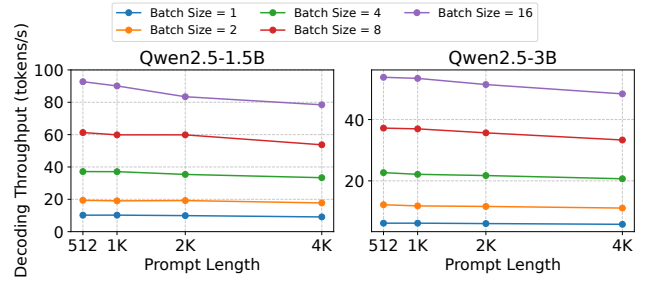


Figure 17. Impact of prompt length on decoding throughput.

Impact of Prompt Lengths. Figure 17 shows the impact of prompt lengths on decoding throughput. Across all batch sizes and both models, the decoding throughput exhibits a mild decreasing trend as the prompt length increases from 512 to 4096 tokens. However, within the range of prompt lengths up to 4096 tokens, this decline remains relatively subtle, indicating that prompt length exerts only a limited influence on decoding throughput in this interval.

8 Discussion

Generalizability to Other Hardwares. We argue that the "vector + matrix" architecture of NPUs possesses a certain degree of universality and observe that the boundary between CPUs and NPUs is gradually blurring. Beyond NPUs, modern CPUs have also begun to incorporate dedicated matrix multiplication units, such as Intel AMX and ARM SME, endowing them with a similar "vector + matrix" architecture. Furthermore, we note that modern AI accelerators generally exhibit a significant disparity between general-purpose computing performance and specialized low-precision matrix multiplication capabilities (e.g. NVIDIA GPUs). Although specific hardware architectures may differ, the core ideas behind our techniques maintain broad applicability.

System Performance and Limitations. (a) Decoding Performance: The current decoding speed of our system is relatively constrained, primarily due to the overhead of dequantization. However, this does not undermine the effectiveness of test-time scaling. Quantized GEMM based on QNN typically utilizes only the DMA and HMX components without introducing HVX computational overhead. Approaches similar to T-MAC [56] could potentially enable efficient GEMV with fine-grained group quantization on NPUs, thereby accelerating the LLM decoding process. (b) Prefill Performance: There remains room for improvement in the prefill performance of our current system. Offloading more operators to the NPU, reducing memory access and communication overhead through operator fusion, and optimizing tiling and pipelining strategies for matrix multiplication could all contribute to enhanced prefill performance. We leave these optimizations to future work. (c) Model Size Constraints: Our current implementation is limited by the 32-bit address space of a single NPU session on older devices. Employing multiple NPU sessions could help alleviate this issue.

Application Scope of Parallel Test-time Scaling. Although parallel test-time scaling methods currently dominate mathematical reasoning tasks, evidence from recent studies [7, 16, 21, 45, 62, 64] indicates their extensibility to broader reasoning and planning domains, highlighting substantial generalizable potential.

9 Related Works

On-Device LLM Inference with NPUs. `llm.npu` [59] pioneered the use of per-tensor quantized INT8 GEMM on NPUs to accelerate the prefill phase of LLMs, employing the CPU to assist in outlier-related computations to maintain accuracy. HeteroLLM [6] achieves collaborative inference between GPUs and NPUs through tensor partitioning. PowerServe [2], an open-source inference framework, leverages the intermediate ONNX format to implement custom quantized and floating-point computation partitioning. ShadowAttn [66] utilizes collaboration between NPUs and CPUs/GPUs to accelerate sparse attention. ExecuTorch [44, 53] is a well-known open-source edge-side DNN inference framework, supporting SpinQuant [36] and the QNN backend. All of the above works are based on Qualcomm’s Hexagon NPU and use the closed-source QNN [47] as the backend. Works from vivo [5, 38] have utilized MediaTek’s NPU, but due to the non-public accessibility of the NeuroPilot SDK, such research remains scarce.

LLM Quantization. The most well-known post-training quantization algorithms include GPTQ [14] and AWQ [32], they perform weight-only quantization and only require small amounts of calibration data, therefore they are extensively used. Subsequent methods such as SmoothQuant [58],

DuQuant [31] broadened the scope to include weight-activation quantization, tackling the more challenging task of quantizing activations by developing techniques to mitigate their problematic outlier distributions. Most recently, comprehensive approaches such as QuaRot [1] and SpinQuant [36] have emerged, aiming to quantize all major components - weights, activations, and the critical KV cache - often down to 4-bits. These methods leverage rotation transformations to create more quantization-friendly feature distributions throughout the model.

Speculative Decoding. Speculative Decoding [27, 37, 41] is a class of acceleration methods for LLM inference, the core of which is to verify multiple speculated tokens in one model forward pass to alleviate the memory-bound issue of LLM decoding. There are various extended variants of Speculative Decoding, and some [18, 29] no longer strictly follow the distribution of the target model. In theory, generalized Speculative Decoding and test-time scaling methods both belong to the generalized Generate-then-Verify framework, and our system can theoretically support these applications seamlessly.

10 Conclusion

This work demonstrates the feasibility and effectiveness of leveraging the underutilized compute capacity of mobile NPUs — specifically the Qualcomm Hexagon NPU — for test-time scaling of LLMs. By designing an end-to-end inference system incorporating hardware-aware tile quantization, weight layout optimization, and LUT-based acceleration of key operators, we show that smaller models augmented with test-time scaling can outperform larger conventionally-deployed models in both accuracy and latency. This approach provides a new pathway for deploying high-performance language models on resource-constrained mobile devices, advancing the Pareto frontier of efficiency and capability for on-device AI.

11 Acknowledgement

We thank all the anonymous reviewers and our shepherd, Yubin Xia, for their insightful feedback and suggestions. This work was supported by Tsinghua University (AIR)-AsiaInfo Technologies (China), Inc. Joint Research Center for 6G Network and Intelligent Computing under Grant 20233910006.

References

- [1] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. *Advances in Neural Information Processing Systems*, 37:100213–100240, 2024.
- [2] PowerServe Authors. Powerserve. <https://github.com/powerserve-project/PowerServe/tree/main>, 2025.
- [3] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys:

- Scaling inference compute with repeated sampling. [arXiv preprint arXiv:2407.21787](#), 2024.
- [4] Jiefeng Chen, Jie Ren, Xinyun Chen, Cheng Run Yang, Ruoxi Sun, and Serkan O. Arik. Sets: Leveraging self-verification and self-correction for improved test-time scaling. [arXiv preprint arXiv:2501.19306](#), 2025.
 - [5] Jiyu Chen, Poh Seng Lim, Shuang Peng, Daxiong Luo, JungHau Foo, Yap Deep, Timothy Lee Jun Jie, Kelvin Teh Kae Wen, Fan Yang, Danyu Feng, et al. Edgeinfinite-instruct: Bridging sft-based optimization and npu-level efficiency for edge devices. [arXiv preprint arXiv:2508.00370](#), 2025.
 - [6] Le Chen, Dahu Feng, Erhu Feng, Rong Zhao, Yingrui Wang, Yubin Xia, Haibo Chen, and Pinjie Xu. Heterollm: Accelerating large language model inference on mobile socs platform with heterogeneous ai accelerators. [arXiv preprint arXiv:2501.14794](#), 2025.
 - [7] Mouxiang Chen, Binyuan Hui, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Jianling Sun, Junyang Lin, and Zhongxin Liu. Parallel scaling law for language models. [arXiv preprint arXiv:2505.10475](#), 2025.
 - [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. [arXiv preprint arXiv:2110.14168](#), 2021.
 - [9] Intel Corporation. Intel npu acceleration library. <https://intel.github.io/intel-npu-acceleration-library/npu.html>, 2024. Accessed: [Insert date here].
 - [10] Intel Corporation. Intel gaudi 3 ai accelerator white paper. <https://www.intel.com/content/www/us/en/content-details/817486/intel-gaudi-3-ai-accelerator-white-paper.html>, 2025. Accessed: 2025.
 - [11] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, July 2023.
 - [12] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. [arXiv preprint arXiv:2305.14314](#), 2023.
 - [13] Yash Dwivedi, Aman Madaan, Uri Alon, Graham Neubig, Kyle Richardson, Wen-tau Yih, Sourav Roy, and Arman Cohan. Making language models better reasoners with step-aware verifier. [arXiv preprint arXiv:2306.04509](#), 2023.
 - [14] Elias Frantar, Saleh Ashkboos, Torsten Hoeftler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. [arXiv preprint arXiv:2210.17323](#), 2022.
 - [15] Georgi Gerganov. llama.cpp: Inference of meta's llama model (and others) in pure c/c++. <https://github.com/ggml-org/llama.cpp>, 2025.
 - [16] Danijar Hafner, Pranav Deka, Shixiang Shane Gu, Timothy Lillicrap, and Mohammad Norouzi. Reasoning as planning: LLMs as building blocks of a rational agent. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
 - [17] Zixu Hao. fastrpc_munmap/remote_mem_unmap does not seem to remove dsp-side mapping immediately #137, 2025.
 - [18] Zixu Hao, Huiqiang Jiang, Shiqi Jiang, Ju Ren, and Ting Cao. Hybrid slm and llm for edge-cloud collaborative inference. In *Proceedings of the Workshop on Edge and Mobile Foundation Models*, pages 36–41, 2024.
 - [19] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. [arXiv preprint arXiv:2009.03300](#), 2020.
 - [20] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. [arXiv preprint arXiv:2103.03874](#), 2021.
 - [21] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Ceyao Lin, Wen-Yi Liu, Bill Yin, David Jiang, Deheng Fu, Zhiyuan Lin, et al. Metagpt: Meta programming for multi-agent collaborative framework. [arXiv preprint arXiv:2308.00352](#), 2023.
 - [22] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. [arXiv preprint arXiv:2404.06395](#), 2024.
 - [23] HuggingFaceH4. Scaling test-time compute - a hugging face space. <https://huggingface.co/spaces/HuggingFaceH4/blogpost-scaling-test-time-compute>, 2025.
 - [24] AMD Inc. Amd processors specifications. <https://www.amd.com/en/products/specifications/processors.html>, 2025. Accessed: 2025.
 - [25] Qualcomm Incorporated. Unlocking on-device generative ai with an npu and heterogeneous computing. <https://www.qualcomm.com/content/dam/qcomm-martech/dm-assets/documents/Unlocking-on-device-generative-AI-with-an-NPU-and-heterogeneous-computing.pdf>, 2024. Accessed: 2025.
 - [26] Oleksandra Kovaleva, Tim Dettmers, Mikel Artetxe, Luke Zettlemoyer, Mike Lewis, Gautier Izacard, and Edouard Grave. Systematic outliers in large language models. [arXiv preprint arXiv:2402.01353](#), 2024.
 - [27] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.
 - [28] Zhen Li, Yupeng Su, Runming Yang, Congkai Xie, Zheng Wang, Zhongwei Xie, Ngai Wong, and Hongxia Yang. Quantization meets reasoning: Exploring llm low-bit quantization degradation for mathematical reasoning. [arXiv preprint arXiv:2501.03035](#), 2025.
 - [29] Bao Hao Liao, Yuhui Xu, Hanze Dong, Junnan Li, Christof Monz, Silvio Savarese, Doyen Sahoo, and Caiming Xiong. Reward-guided speculative decoding for efficient llm reasoning. [arXiv preprint arXiv:2501.19324](#), 2025.
 - [30] Heng Liao, Jiajin Tu, Jing Xia, Hu Liu, Xiping Zhou, Honghui Yuan, and Yuxing Hu. Ascend: a scalable and unified architecture for ubiquitous deep neural network computing: Industry track paper. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 789–801. IEEE, 2021.
 - [31] Haokun Lin, Haobo Xu, Yichen Wu, Jingzhi Cui, Yingtao Zhang, Linzhan Mou, Linqi Song, Zhenan Sun, and Ying Wei. Duquant: Distributing outliers via dual transformation makes stronger quantized llms. *Advances in Neural Information Processing Systems*, 37:87766–87800, 2024.
 - [32] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024.
 - [33] Yu-Shan Lin, Cheng-En Wu, Hsin-Hsuan Chen, Chi-Jen Lee, and Da-Cheng Juan. Do emergent abilities exist in quantized large language models: An empirical study. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 14076–14087. Association for Computational Linguistics, 2023.
 - [34] Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization hurts reasoning? an empirical study on quantized reasoning models. [arXiv preprint arXiv:2504.04823](#), 2025.
 - [35] Runze Liu, Junqi Gao, Jian Zhao, Kaiyan Zhang, Xiu Li, Biquing Qi, Wanli Ouyang, and Bowen Zhou. Can 1b llm surpass 405b llm? rethinking compute-optimal test-time scaling. [arXiv preprint arXiv:2502.06703](#), 2025.
 - [36] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. Spinqant: Llm quantization with learned rotations. [arXiv preprint arXiv:2405.16406](#), 2024.
 - [37] Xiaofan Lu, Yixiao Zeng, Feiyang Ma, Zixu Yu, and Marco Levorato. Improving multi-candidate speculative decoding. [arXiv preprint arXiv:2409.10644](#), 2024.
 - [38] Xudong Lu, Yinghao Chen, Cheng Chen, Hui Tan, Boheng Chen, Yina Xie, Rui Hu, Guanxin Tan, Renshou Wu, Yan Hu, et al. Bluelm-v-3b:

- Algorithm and system co-design for multimodal large language models on mobile devices. [arXiv preprint arXiv:2411.10640](#), 2024.
- [39] Eric Mahurin. Qualcomm® hexagon™ npu. In *2023 IEEE Hot Chips 35 Symposium (HCS)*, pages 1–19, 2023.
- [40] Meta. Llama 3.2: Vision and edge-optimized models for multimodal and mobile ai. Meta AI Blog, 2024.
- [41] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949, 2024.
- [42] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. [arXiv preprint arXiv:1805.02867](#), 2018.
- [43] o1 Team Skywork. Skywork-o1 open series. <https://huggingface.co/Skywork>, 2024.
- [44] Andrew Or, Apurva Jain, Daniel Vega-Myhre, Jesse Cai, Charles David Hernandez, Zhenrui Zheng, Driss Guessous, Vasily Kuznetsov, Christian Puhersch, Mark Saroufim, et al. Torchao: Pytorch-native training-to-serving model optimization. [arXiv preprint arXiv:2507.16099](#), 2025.
- [45] Bhavana Paranjape, Amit Budhiraja, Amir Stdeghani, Gholamreza Haffari, and Sameer Sawhney. Boosting llm reasoning: A crossover of tree of thoughts and retrieve-augmented generation. [arXiv preprint arXiv:2310.00844](#), 2023.
- [46] Qualcomm Innovation Center, Inc. fastrpc: Fastrpc library for linux userspace. <https://github.com/quic/fastrpc>, 2025.
- [47] Inc. Qualcomm Technologies. Qualcomm® ai engine direct sdk. <https://developer.qualcomm.com/software/qualcomm-ai-engine-direct-sdk>, 2025. Unified API for AI development on Qualcomm accelerators (Hexagon DSP, Kryo CPU, Adreno GPU). Supports direct offloading for TensorFlow Lite/ONNX Runtime, with HTP (Hexagon Tensor Accelerator) and CDSP (Hexagon Compute DSP) backend optimizations. Accessed: 2025-05-16.
- [48] Alejandro Rico, Satyaprakash Pareek, Javier Cabezas, David Clarke, Baris Ozgul, Francisco Barat, Yao Fu, Stephan Münz, Dylan Stuart, Patrick Schlangen, et al. Amd xdna™ npu in ryzen™ ai processors. *IEEE Micro*, 2024.
- [49] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. [arXiv preprint arXiv:1907.10641](#), 2019.
- [50] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. [arXiv preprint arXiv:2408.03314](#), 2024.
- [51] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. Gemma 3 technical report. [arXiv preprint arXiv:2503.19786](#), 2025.
- [52] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. [arXiv preprint arXiv:2403.08295](#), 2024.
- [53] PyTorch Team. Executorch: A pytorch platform for on-device deployment. <https://github.com/pytorch/executorch>, 2025. Accessed: 2025.
- [54] Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. [arXiv preprint arXiv:2312.08935](#), 2023.
- [55] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. [arXiv preprint arXiv:2303.11366](#), 2023.
- [56] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. T-mac: Cpu renaissance via table lookup for low-bit llm deployment on edge. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 278–292, 2025.
- [57] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. [arXiv preprint arXiv:2408.00724](#), 2024.
- [58] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR, 2023.
- [59] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. Fast on-device llm inference with npus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 445–462, 2025.
- [60] Fengli Xu, Qianyu Hao, Zefang Zong, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, Tianjian Ouyang, Fan Jinmeng, et al. Towards large reasoning models: A survey on scaling llm reasoning capabilities. [arXiv preprint arXiv:2501.09686](#), 2025.
- [61] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. Powerinfer-2: Fast large language model inference on a smartphone. [arXiv preprint arXiv:2406.06282](#), 2024.
- [62] Yuxuan Yan, Shiqi Jiang, Ting Cao, Yifan Yang, Qianqian Yang, Yuanhao Shu, Yuqing Yang, and Lili Qiu. Empowering agentic video analytics systems with video language models. [arXiv preprint arXiv:2505.00254](#), 2025.
- [63] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. [arXiv preprint arXiv:2412.15115](#), 2024.
- [64] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- [65] Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, et al. Minicpm-v: A gpt-4v level mllm on your phone. [arXiv preprint arXiv:2408.01800](#), 2024.
- [66] Wangsong Yin, Daliang Xu, Mengwei Xu, Gang Huang, and Xuanzhe Liu. Dynamic sparse attention on mobile socs. [arXiv preprint arXiv:2508.16703](#), 2025.
- [67] Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Process-bench: Identifying process errors in mathematical reasoning. [arXiv preprint arXiv:2412.06559](#), 2024.